

Processing ECG Time-Series Data with Polars

After the COVID-19 pandemic, human health has become a top priority. COVID-19 not only affects the respiratory system but also causes severe complications related to cardiovascular health, the nervous system, and mental well-being. Many people experience post-COVID conditions such as reduced lung function, irregular heartbeats, or prolonged stress. As a result, the demand for personal health monitoring has increased significantly, driving the rapid development of health-tracking devices.

As health monitoring devices continue to evolve, one of the most critical signals in cardiovascular assessment is the electrocardiogram (ECG), which plays a vital role in detecting heart abnormalities. However, collecting and analyzing ECG data presents significant challenges due to the large volume of data involved.

To assess heart rate conditions, it is essential to track key ECG features such as the QRS complex, T wave, and P wave. These elements provide deeper insights into heart rhythm and cardiovascular health, enabling more accurate monitoring and diagnosis. This allows for more comprehensive monitoring and diagnosis, offering a clearer picture of any potential abnormalities or issues in heart function.

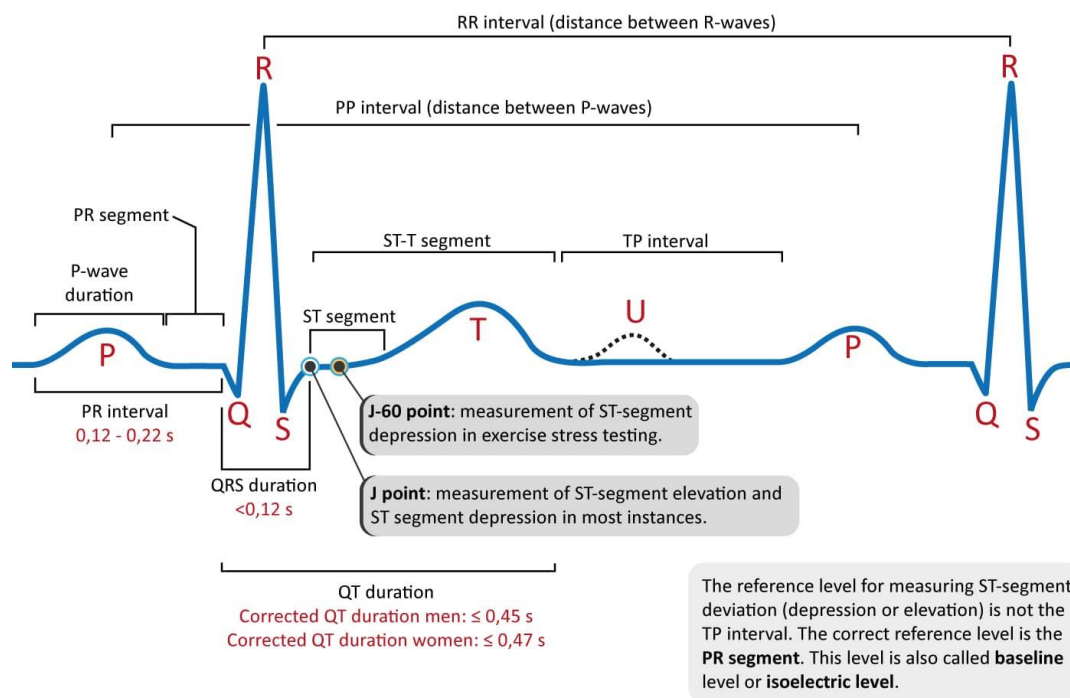


Figure 1. The classical ECG curve with its most common waveforms. Important intervals and points of measurement are depicted. ECG interpretation requires knowledge of these waves and intervals.

What is Polars?

The healthcare device records data every hour and conducts long-term monitoring for up to 30 days, enabling the storage of data for statistical analysis and summary of the Holter reports. It captures all specifications of each QRS complex, detecting any anomalies. Each characteristic of the QRS complex plays a crucial role in identifying abnormalities or potential health conditions in the user.

To manage the demands of large-scale ECG data storage and processing, Polars emerges as a highly efficient framework. Some of its key advantages include:

First and foremost, Polars is primarily written in **Rust**—a language designed for high performance and safety, operating close to "**the metal**". This architecture grants Polars a significant performance edge, as Rust allows precise control over critical aspects such as memory management and CPU multithreading. Additionally, Rust enhances software reliability by detecting potential issues at compile time rather than permitting runtime errors. Another key advantage of Polars' Rust-based foundation is its cross-language compatibility, making it seamlessly usable in both Python and Rust environments.

Secondly, Polars is built on **Apache Arrow**, a framework that provides a standardized, column-oriented memory structure optimized for in-memory computing. Arrow's memory format is specifically designed for high-efficiency operations on both flat and hierarchical data structures, addressing performance limitations commonly found in Pandas, particularly in data ingestion and export.

One of Apache Arrow's standout features is its ability to memory-map IPC files locally. This enables efficient handling of datasets that exceed available memory while facilitating seamless data sharing across languages and processes. The focus here is not just on processing large datasets, but also on optimizing data accessibility and interoperability.

In addition, Polars is optimized for **minimal memory** usage, making it ideal for working with massive datasets that may not fit entirely into RAM. This memory efficiency ensures Polars can handle large-scale data with minimal overhead while maximizing performance.

Moreover, Polars supports advanced data types such as **datetime64** and **timedelta64**, which are not natively supported by Pandas. These advanced types provide greater precision and flexibility for **time-series analysis** and **date manipulations**, making Polars particularly advantageous for applications requiring extensive time-based computations. In our system, ECG signals are recorded and stored as continuous time-series data while the device is being worn, with each data point marked by epoch time.

This design offers a significant advantage for applying the Polars framework, which is highly optimized for time-series data processing and supports efficient filtering, aggregation, and time-based operations at scale.

Finally, Polars can leverage **GPU** acceleration for specific DataFrame operations, significantly enhancing performance in computationally intensive tasks. This GPU support further strengthens Polars' capabilities in large-scale data processing, enabling faster execution times compared to traditional CPU-based processing.

Benchmark Performance

Dataset Description

To benchmark the performance of Polars against other libraries, we use ECG data collected and saved as a Parquet file. The DataFrame contains approximate 10 million rows and 25 columns, including details such as beat position, beat types, QRS specifications, and attributes like P/T wave onset, offset, peak, amplitude, ST level, slope, and more.

shape: (10_283_379, 25)

EPOCH	CHANNEL	BEAT	BEAT_TYPE	-	PVC_TEMPLA	PVC_TEMPLA	PVC_TEMPLA	IS_UPDATED
---	---	---	---		TE	TE_CORRCOE	TE_REVIEWE	---
164	164	164	164		---	F	0	bool
					164	---	---	
						164	164	
1741104103	17	183	1	-	0	0	0	false
732				-	0	0	0	false
1741104104	17	364	1	-	0	0	0	false
456				-	0	0	0	false
1741104105	17	541	1	-	0	0	0	false
164				-	0	0	0	false
1741104105	17	787	1	-	0	0	0	false
828				-	0	0	0	false
1741104106	17	866	1	-	0	0	0	false
464				-	0	0	0	false
-	-	-	-	-	-	-	-	-
1743784760	577	24752	1	-	0	0	0	false
452				-	0	0	0	false
1743784760	577	24976	1	-	0	0	0	false
560				-	0	0	0	false
1743784760	577	25210	1	-	0	0	0	false
548				-	0	0	0	false
1743784760	577	25454	1	-	0	0	0	false
548				-	0	0	0	false
1743784760	577	25690	1	-	0	0	0	false
476				-	0	0	0	false

Figure 2. The ECG DataFrame

In this blog, we conduct performance evaluations across different versions and system configurations, as shown in the tables below.

Table 1: Benchmark Configuration Table

COMPONENT	SPECIFICATION
OS	Ubuntu 22.04.3 LTS
CPU	AMD® Ryzen 5 3600 (6 cores, 12 threads)
RAM	64,0 GB DDR4
STORAGE	NVMe SSD 500 GB
PYTHON VERSION	3.11.10
POLARS VERSION	1.6.0
PANDAS VERSION	2.2.3
NUMPY VERSION	1.26.4

(*) To achieve a fair and reliable comparison, the mean processing time across 10 iterations for each query method will smooth out any performance fluctuations due to system load or caching.

Read Parquet File

Comparing reading time of the dataset

+-----+ READ +-----+				
packages	minTime (s)	avgTime (s)	maxTime (s)	
polars	0.306834	0.332887	0.417788	
pandas	1.588548	1.662955	1.917322	
numpy	0.502786	0.735573	1.231828	

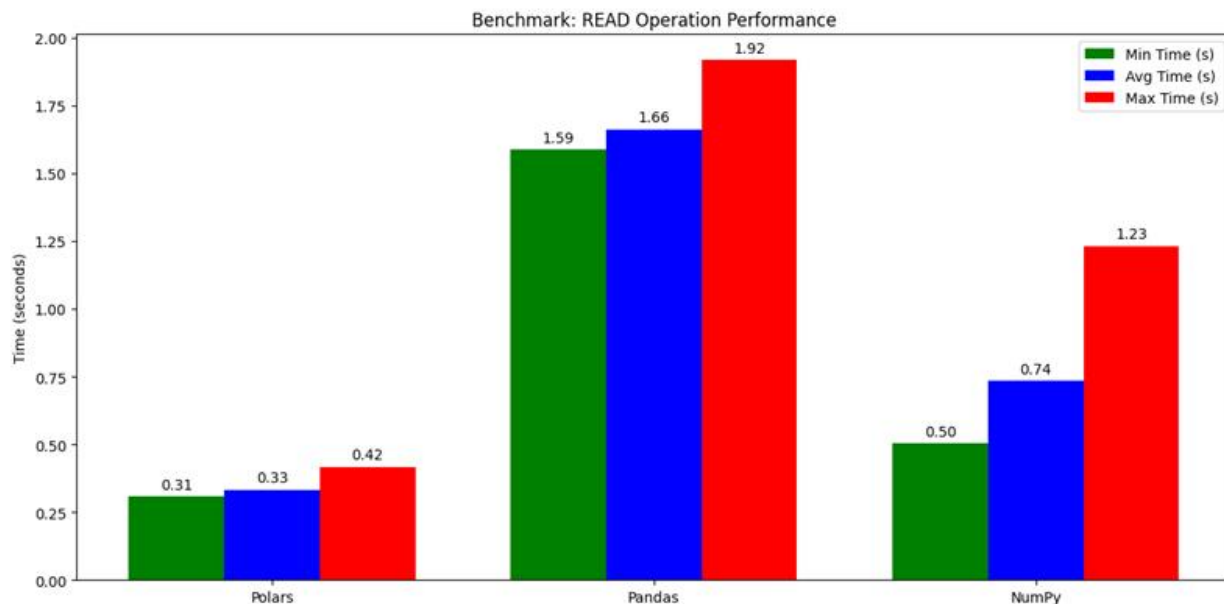


Figure 4. Benchmark: READ

Using **Polars**, it takes only **0.3328 seconds** to read a dataset of Parquet files.

In comparison:

- **Pandas** takes **1.6629** seconds – making it approximately 5× slower than Polars.
- **NumPy** takes **0.7356** seconds – still over 2× slower and less efficient for structured data loading.

This demonstrates the superior performance of Polars in terms of **I/O speed and memory efficiency**, particularly when working with columnar formats like **Parquet**.

Writing to Parquet Files & Output Size

When writing a large dataset to Parquet files, the performance and storage efficiency of each library differ significantly:

Table 2: Statistical

LIBRARY	WRITE TIME (S)	FILE SIZE
POLARS	7.0371	271 MB
PANDAS	9.5957	625 MB
NUMPY	2.6191	2.1 GB

WRITE			
packages	minTime (s)	avgTime (s)	maxTime (s)
polars	6.846136	7.037001	7.594364
pandas	7.713581	9.59571	13.39587
numpy	2.520703	2.619046	2.71941

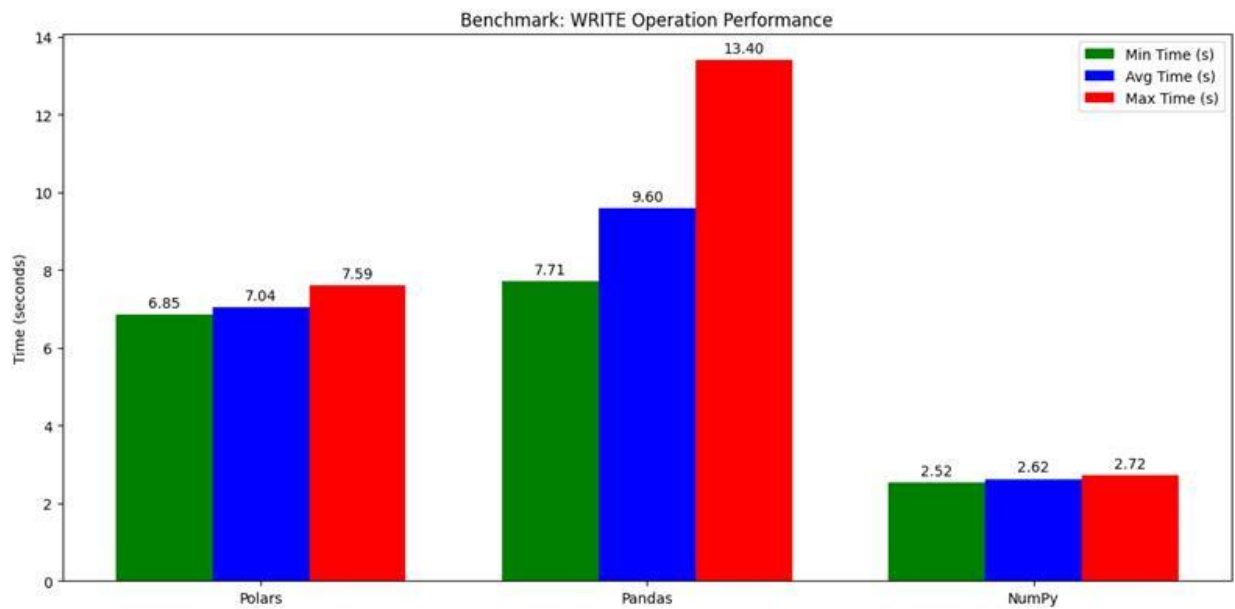


Figure 5. Benchmark: WRITE

Name	Size	Modified
data-test-numpy.npy	2,1 GB	14:42
data-test-pandas.parquet	625,0 MB	14:42
data-test-polars.parquet	271,2 MB	14:42

Figure 6. File sizes

Key observations:

- **NumPy** writes the fastest but produces the **largest file size**, since it lacks support for compression and data structure optimization in Parquet format.
- **Pandas** is slower and generates a file that is more than twice the size of Polars.

- **Polars** strikes the **best balance**, offering **efficient write speed** and **minimal output size**, thanks to Arrow's columnar memory format and internal compression.

File size has a **direct impact** on the system's performance, especially when working in cloud environments. In our system, all ECG data is stored on **AWS S3**, and the **smaller the file**, the **faster the upload/download process**. This significantly reduces:

- Data transmission time over the network
- S3 read/write latency
- Costs related to bandwidth and storage

Polars' ability to produce **compact and optimized files** gives it a crucial edge in real-world deployments where speed and cost-efficiency are priorities.

Querying / Filtering Data

QUERY				
packages	minTime (s)	avgTime (s)	maxTime (s)	
polars	0.004407	0.006232	0.019061	
pandas	0.036635	0.040271	0.047227	
numpy	0.007594	0.00792	0.008713	

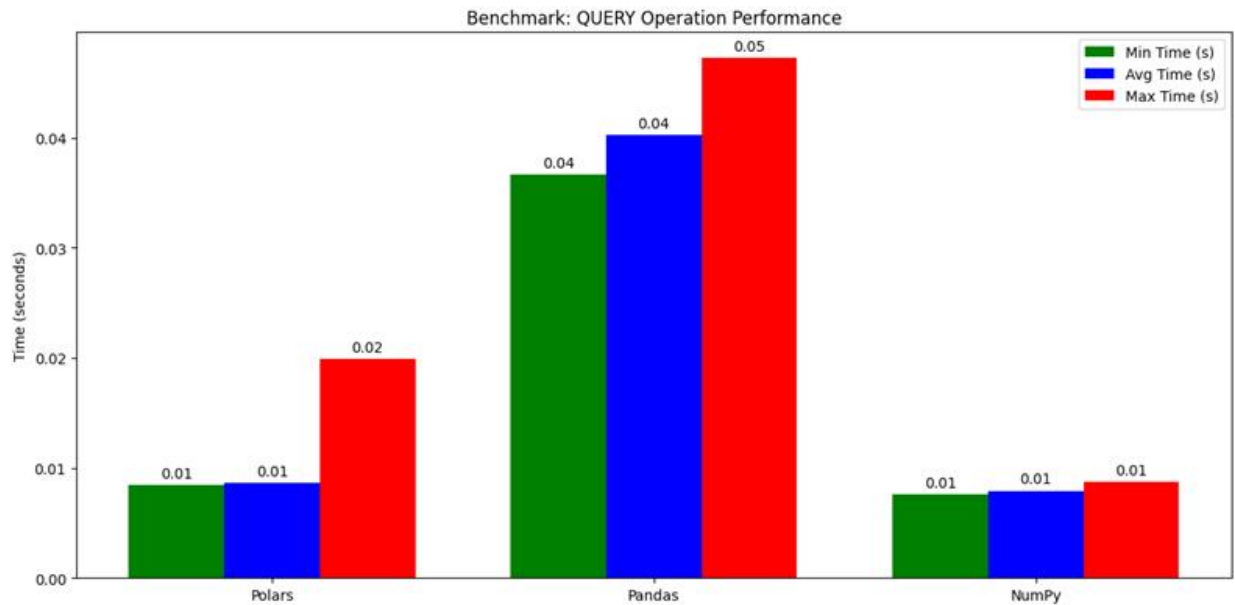


Figure 7. Benchmark: QUERY

When querying and filtering the DataFrame—such as selecting rows based on conditions like calculating RR intervals—Polars achieved an **average execution time of 0.0362s**, which is **slightly faster than Pandas (0.0403s)** and **marginally faster than NumPy (0.0079s)**.

Although NumPy appears faster in absolute time, it's worth noting that NumPy typically requires more **verbose and lower-level syntax**, making it harder to express complex filtering logic clearly. In contrast, Polars provides a **clean and expressive syntax** for queries (e.g., using `.filter()` or `.when().then()` chains), giving developers the **best balance of performance and readability**.

Sorting Data

SORT			
packages	minTime (s)	avgTime (s)	maxTime (s)
polars	1.980771	2.059136	2.434868
pandas	2.041358	2.477743	4.909177
numpy	6.791565	6.856815	6.933506

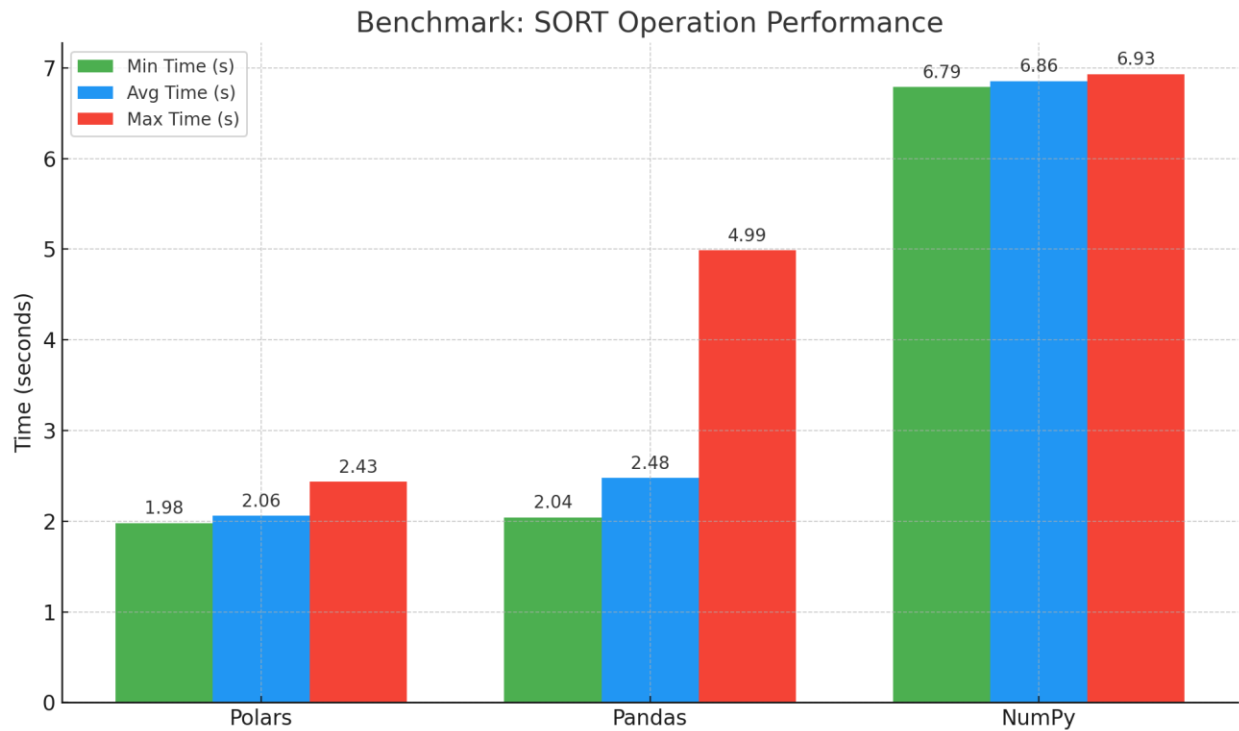


Figure 8. Benchmark: SORT

Polars consistently outperforms the others with the lowest average sort time (~2.06s). **Pandas** performs moderately but shows high variability, with a max time over $2\times$ higher than Polars. **NumPy** is significantly slower across all metrics, taking $\sim 3\times$ longer than Polars on average.

Update Data

UPDATE			
packages	minTime (s)	avgTime (s)	maxTime (s)
polars	0.012232	0.01568	0.042994
pandas	2.211707	2.235329	2.266898
numpy	0.025181	0.025907	0.027312

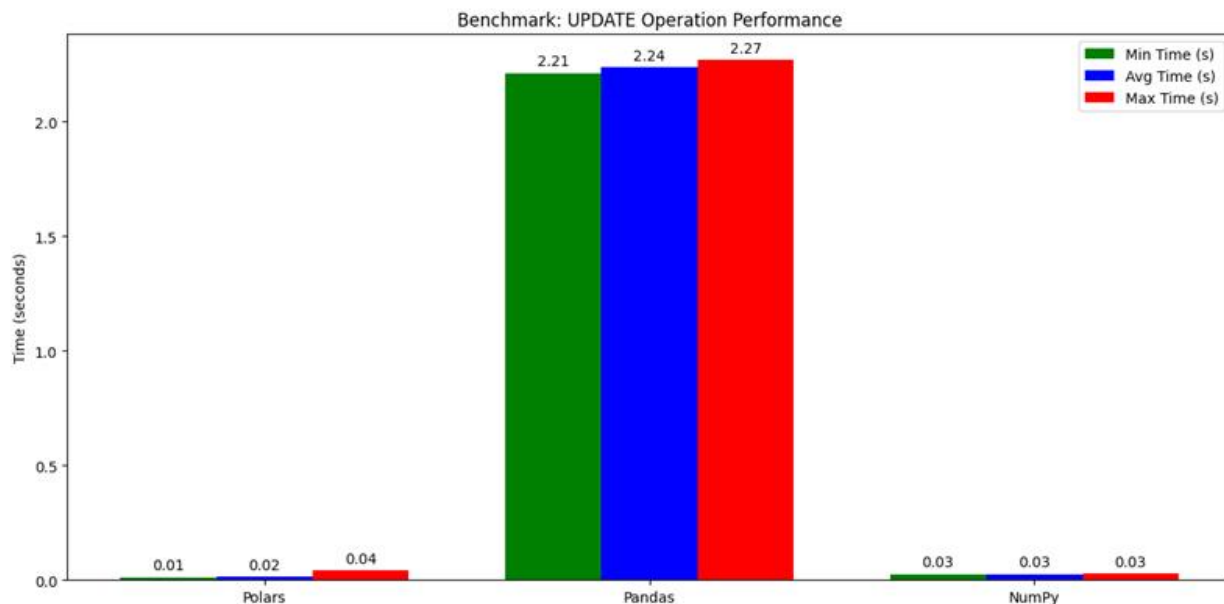


Figure 9. Benchmark: UPDATE

In the update operation—where each row in the DataFrame was conditionally updated—**Polars** once again showed outstanding performance with an **average time of only 0.0157s**, significantly faster than **Pandas** with **2.235s**. **NumPy** also performed well in this task, clocking an average of 0.0259s, though it lacks the same high-level syntax readability as Polars.

These results highlight how Polars excels in efficient data manipulation, especially in workflows that involve conditional updates or large-scale transformations. While Pandas remains popular, its performance on update tasks is noticeably slower due to internal inefficiencies when dealing with larger datasets.

Conclusion

Through our benchmarks on real-world ECG datasets, Polars consistently demonstrates impressive performance across all operations—from reading, writing, sorting, querying, to updating data. It outperforms Pandas in speed and efficiency, especially in read and sort operations, while also offering significantly smaller file sizes (e.g., 271MB vs 625MB for the same data). Although NumPy shows good performance in some basic operations, it lacks the expressiveness and scalability for complex data pipelines.

For systems working with time-series health data (like ECG) and deployed on cloud environments (e.g., AWS S3), smaller file sizes and fast execution times are critical. Polars offers both, making it an excellent candidate for high-performance and cloud-optimized data processing.